

ROSSMANN STORE SALES FORECASTING

End-to-End Machine Learning, Deep Learning & Web Prediction Project

FINAL PROJECT REPORT

Prepared according to the supplied Project 6 PDF instructions

Workflow: EDA → ML Pipeline → LSTM → MLflow → Streamlit

1. Executive Summary

This project develops an end-to-end Rossmann store sales forecasting solution. The business objective is to forecast daily sales up to six weeks ahead and support finance teams and store managers with data-driven planning. The supplied project specification requires exploration of customer purchasing behaviour, machine-learning prediction, deep learning, model serving and a web interface.

The project combines data cleaning, feature engineering, exploratory analysis, scikit-learn pipelines, tree-based regression, uncertainty estimation, a two-layer LSTM, MLflow tracking and a Streamlit dashboard.

2. Business Problem

Store managers traditionally use experience and personal judgement to forecast sales. The proposed system replaces this manual-only process with a reproducible forecasting workflow using historical sales, customers, promotions, holidays, store characteristics and competition information.

3. Project Objectives

- Explore customer purchasing behaviour.
- Analyse promotion, holiday, seasonal and competition effects.
- Build an ML regression pipeline for store sales prediction.
- Evaluate models using RMSE, MAE, R^2 and RMSPE.
- Estimate prediction uncertainty.
- Build a two-layer LSTM time-series model.
- Track experiments with MLflow.
- Serve predictions through Streamlit and export CSV results.

4. Dataset Overview

Dataset	Rows	Columns	Role
train.csv	1,017,209	9	Historical labelled data
test.csv	41,088	8	Prediction data
store.csv	1,115	10	Store metadata

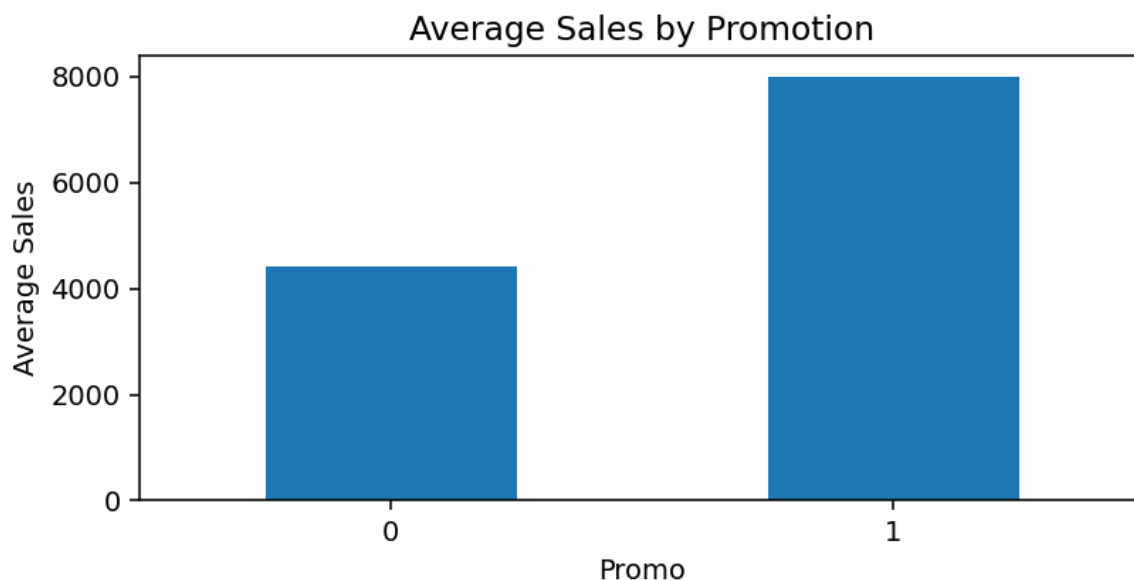
5. Data Cleaning & Feature Engineering

Date is converted to datetime and store metadata is merged using Store. The workflow audits missing values and duplicates, handles numeric and categorical variables through a modelling pipeline, and derives Year, Month, Day, WeekOfYear, DayOfWeek, IsWeekend, month-position indicators and competition-age features.

6. Exploratory Data Analysis

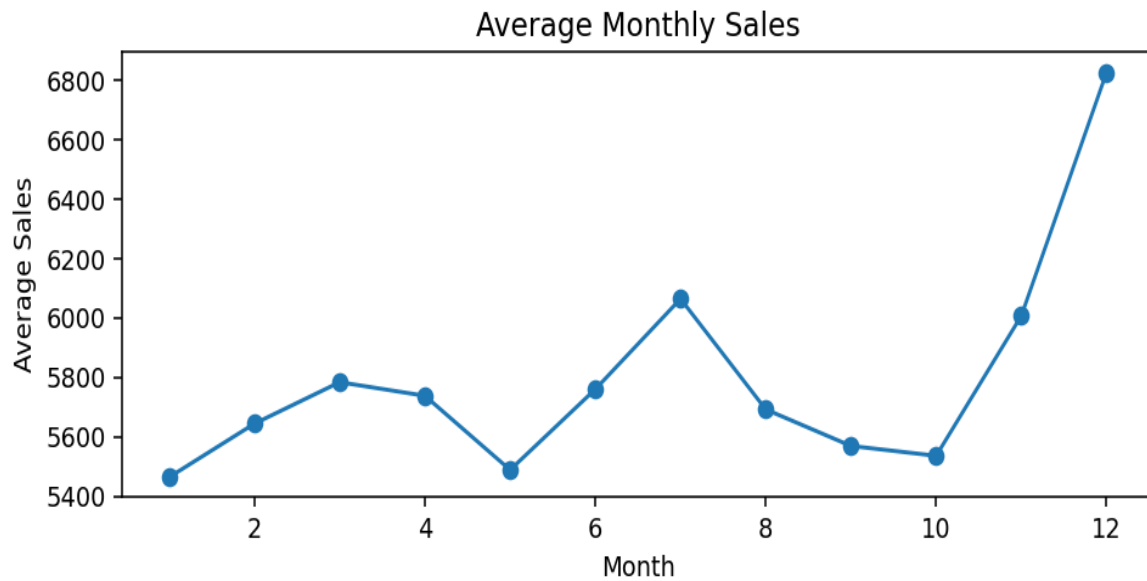
Average training sales are approximately **5,774** and average customers approximately **633**. Sales and Customers show a correlation of **0.895** in the supplied training data.

Promotion appears in approximately **38.2%** of training rows and **39.6%** of test rows.

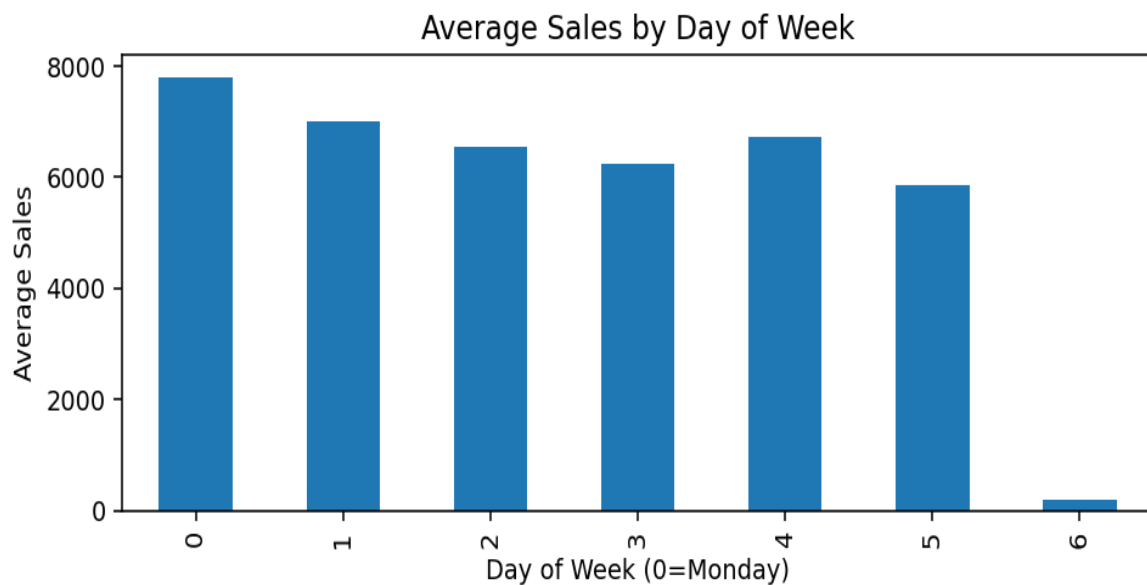


Raw group comparison shows average sales during promotion are approximately **81.4%** higher than non-promotion days. This is an observational association and should not be interpreted as causal impact.

7. Seasonality & Calendar Behaviour



Monthly trends are inspected to identify seasonal purchasing behaviour. The notebook also evaluates holiday effects and store behaviour across open/closed days.



Weekday/weekend behaviour is analysed using day-of-week features. Additional EDA covers assortment type, competition distance and competitor opening/reopening patterns.

8. Machine Learning Methodology

Sales forecasting is formulated as a regression problem. A time-aware validation split is used to reduce leakage from future observations. The scikit-learn pipeline uses median imputation and StandardScaler for numerical features, and most-frequent imputation plus OneHotEncoder for categorical features.

Models: Random Forest Regressor and Extra Trees Regressor.

Metrics: RMSE, MAE, R^2 and RMSPE. RMSE penalizes larger errors; MAE is directly interpretable; R^2 measures explained variance; RMSPE gives a relative error perspective.

Exact validation scores are generated by executing the supplied notebook; no model scores are fabricated in this report.

9. Feature Importance & Confidence Interval

Feature importance is extracted from the fitted tree model. An empirical 95% prediction interval is estimated from validation residual variability. This is a practical uncertainty estimate, not a formal probabilistic guarantee.

10. Model Serialization

The workflow serializes trained models with timestamps and also saves a final model for application inference, supporting reproducibility and model-version tracking.

11. Deep Learning — Two-Layer LSTM

The deep-learning workflow isolates daily aggregate sales as a time series, checks stationarity using the Augmented Dickey-Fuller test, uses differencing where appropriate for time-series diagnostics, examines ACF/PACF, converts the series into sliding-window supervised data, scales values to (-1, 1), and trains a two-layer LSTM regression model.

Architecture: LSTM(64, return_sequences=True) → Dropout(0.2) → LSTM(32) → Dropout(0.2) → Dense(1). Early stopping is used to reduce overfitting.

12. MLflow

MLflow records the Random Forest model, important hyperparameters and validation metrics. This supports experiment comparison and reproducibility.

13. Streamlit Web Application

The Streamlit application loads the serialized model, accepts a prediction CSV, generates date-derived features, predicts sales, displays a sales trend chart and allows users to download the prediction output as CSV.

14. Project Structure

Component	Purpose
data/	Supplied train, test, store and sample submission data
notebooks/rossmann_end_to_end.ipynb	EDA, ML, LSTM and MLflow workflow
models/	Serialized ML/LSTM models
reports/	Prediction outputs and report
app/streamlit_app.py	Prediction dashboard
requirements.txt	Required Python libraries
README.md	Setup and execution instructions

15. Execution

Install dependencies using **pip install -r requirements.txt**, execute the notebook cells in order, generate the final serialized model, and then run **streamlit run app/streamlit_app.py**. MLflow can be opened with **mlflow ui --backend-store-uri ./mlruns**.

16. Final Submission Checklist

✓ EDA notebook ✓ ML pipeline ✓ LSTM model ✓ MLflow tracking ✓ Serialized models ✓ Streamlit dashboard ✓ CSV prediction output ✓ GitHub-ready structure ✓ Final report

17. Conclusion

The project provides a reproducible sales forecasting pipeline aligned with the supplied Project 6 requirements. It integrates business-focused EDA, feature engineering, machine learning, time-series deep learning, experiment tracking and a practical web interface for prediction delivery.

Report prepared from the supplied Project 6 instructions and uploaded Rossmann datasets. Exact trained-model performance is produced when the notebook is executed.